

# AI Model API Usage Guide

This guide will walk you through the process of making API calls to the two AI models deployed on Huawei Cloud ModelArts for your use in the competition:

1. Qwen3-32B
2. DeepSeek-v3.1

Both models are powerful, text-based generative AI models capable of tasks like question answering, code generation, text summarization, and more.

## 1 Prerequisites: Before You Begin

To use these APIs, you will need the following information, which should have been provided to you by the competition organizers:

- **Model Endpoint URL:** The specific HTTP URL.
- `https://api-ap-southeast-1.modelarts-maas.com/v1/chat/completions`
- **Authentication Token (API Key):**
  - A token to authenticate your requests. This is likely provided in the header (e.g., `x-Auth-Token` or `Authorization: Bearer <token>`).

- **Deployed Model Name**

- You can specify which model you are going to use by specifying the model name in the request body (there is an example in section 2.1)
- The names of the deployed model are as follows:
  - deepseek-v3.1 (DeepSeek's Model)
  - qwen3-32b (Alibaba's Qwen Model)

**⚠ Important:** To get the API key you need to send a request via email to this address

[hwcdeveloper@huawei.com](mailto:hwcdeveloper@huawei.com). The email subject should be "API Key Request" , and the email body should include **team name and brief introduction** about the project.

## 2 Core API Structure

Both models use a similar **HTTP POST** request structure to their respective endpoints. The request body is in JSON format and contains your input prompt and parameters to control the model's generation.

### 2.1 Base Request Format

**Method:** POST

**URL:** `https://api-ap-southeast-1.modelarts-maas.com/v1/chat/completions`

**Headers:**

`Content-Type:` application/json

`X-Auth-Token:` YOUR\_AUTH\_TOKEN\_HERE

Or

`Authorization:` Bearer YOUR\_TOKEN

Body (JSON):

```
{
  "model": "deepseek-v3.1" or "qwen3-32b",
  "messages": [
    {
      "role": "user",
      "content": "Give me a brief intro about AI"
    }
  ]
}
```

2.2 Key Request Parameters

| Parameter  | Type    | Description                                                             | Recommended Value                                 |
|------------|---------|-------------------------------------------------------------------------|---------------------------------------------------|
| prompt     | string  | <b>Required.</b> The input text you want to process.                    | "Write a Python function to calculate factorial." |
| max_tokens | integer | The maximum number of tokens (words/sub-words) the model will generate. | 1024                                              |

| Parameter                | Type                 | Description                                                                                                           | Recommended Value                                            |
|--------------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>temperature</code> | <code>float</code>   | Controls randomness:<br>Lower = more deterministic,<br>Higher = more creative/random.                                 | <code>0.1</code> (precise)<br>to <code>0.8</code> (creative) |
| <code>top_p</code>       | <code>float</code>   | Nucleus sampling: Only the most probable tokens with cumulative probability $\geq$ <code>top_p</code> are considered. | <code>0.9</code>                                             |
| <code>top_k</code>       | <code>integer</code> | Limits sampling to the top K most probable tokens.                                                                    | <code>40</code>                                              |
| <code>stream</code>      | <code>boolean</code> | If <code>true</code> , responses are streamed back incrementally.                                                     | <code>false</code> (for simple use)                          |

### 3 Example API Calls

Here are practical examples for both models in different programming languages.

## Example 1: Basic Call using `curl` (Command Line)

This is a quick way to test the API from your terminal.

### For qwen3-32b:

Calling **Qwen** for Code Generation

```
curl -X POST "https://api-ap-southeast-1.modelarts-maas.com/v1/chat/completions" \  
-H "Content-Type: application/json" \  
-H "X-Auth-Token: YOUR_ACTUAL_TOKEN" \  
-d '{  
    "model": "qwen3-32b",  
    "messages": [  
        {"role": "user", "content": "Write a Python function to check if a number  
is prime."}  
    ],  
    "max_tokens": 300,  
    "temperature": 0.1  
}'
```

### For DeepSeek-V3.1:

Calling deepseekk-v3.1 for Sentiment Analysis

```
curl -X POST "https://api-ap-southeast-1.modelarts-maas.com/v1/chat/completions" \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer YOUR_ACTUAL_TOKEN" \  
-d '{  
    "model": "deepseek-v3.1",  
    "messages": [  
        {  
            "role": "user",  
            "content": "Classify the sentiment of this text: '\''I am so excited about the co  
mpetition!\'\'' . Choose from: Positive, Negative, Neutral."  
        }  
    ],  
    "max_tokens": 300,  
    "temperature": 0.1  
}'
```

```
"max_tokens": 10,  
"temperature": 0.1  
'}
```

## Example 2: Python Code using the `requests` library

This is the most common way to integrate the API into your competition project.

```
python  
  
import requests  
import json  
  
# ===== CONFIGURATION - REPLACE WITH YOUR DETAILS =====  
API_BASE_URL = "https://api-ap-southeast-1.modelarts-maas.com/v1/chat/completions" # Provided in the user guide document  
API_PATH = "/v1/chat/completions"  
API_URL = f"{API_BASE_URL}{API_PATH}"  
AUTH_TOKEN = "YOUR_ACTUAL_TOKEN" # Keep this secret!  
  
headers = {  
    "Content-Type": "application/json",  
    "Authorization": f"Bearer {AUTH_TOKEN}" # Verified authentication method  
}  
  
def call_competition_model(model_name, user_prompt, system_prompt=None, **generation_params):  
    """Calls the competition model API with the verified schema.  
  
    Args:  
        model_name (str): Name of the model ('qwen3-32b' or 'deepseek-v3.1').  
        user_prompt (str): The main instruction or question from the user.  
        system_prompt (str, optional): A system message to set the model's behavior.  
        **generation_params: Additional parameters like max_tokens, temperature, top_p.  
  
    Returns:  
        str: The generated text from the model, or None if the request failed.  
    """  
  
    # Build the messages array
```

```
messages = []
if system_prompt:
    messages.append({"role": "system", "content": system_prompt})
messages.append({"role": "user", "content": user_prompt})

# Build the payload
payload = {
    "model": model_name,
    "messages": messages,
    **generation_params # Unpack any additional parameters
}

try:
    print(f"Sending request to {model_name}...")
    response = requests.post(API_URL, headers=headers, json=payload, timeout=60)
    response.raise_for_status() # Raises an exception for bad HTTP status codes

    # Parse the JSON response
    response_data = response.json()

    # Extract the generated text from the confirmed response structure
    # The output is in: ['choices'][0]['message']['content']
    generated_text = response_data['choices'][0]['message']['content']
    return generated_text.strip()

except requests.exceptions.RequestException as e:
    print(f"API Request failed: {e}")
    if hasattr(e, 'response') and e.response is not None:
        print(f"Status Code: {e.response.status_code}")
        print(f"Error Response: {e.response.text}")
    return None
except (KeyError, IndexError) as e:
    print(f"Failed to parse the API response: {e}")
    print(f"Response JSON: {json.dumps(response_data, indent=2)}")
    return None

# --- Example Usage ---

# Example 1: Simple question to Distil-Llama
print("--- Example 1: Simple Query ---")
```

```
answer = call_competition_model(
    model_name="deepseek-v3.1",
    user_prompt="What is the capital of Germany?",
    max_tokens=50,
    temperature=0.1
)
if answer:
    print(f"Model Answer: {answer}")

# Example 2: Complex task with a system prompt for DeepSeek
print("\n--- Example 2: Complex Task with System Prompt ---")
code = call_competition_model(
    model_name="qwen3-32b",
    system_prompt="You are a helpful and precise coding assistant. Provide only the requested code without any additional explanation.",
    user_prompt="Write an efficient Python function to calculate the Fibonacci sequence up to the nth number.",
    max_tokens=400,
    temperature=0.3
)
if code:
    print(f"Generated Code:\n{code}")

# Example 3: Multi-turn conversation simulation
print("\n--- Example 3: Multi-Turn Conversation ---")
# First, ask a question
history = [
    {"role": "user", "content": "Who wrote the play 'Hamlet'?"}
]
# (You would simulate the first response here and add it to history)
# history.append({"role": "assistant", "content": "William Shakespeare."})

# Then, ask a follow-up question in context
follow_up_answer = call_competition_model(
    model_name="qwen3-32b",
    user_prompt="What is another famous tragedy he wrote?",
    max_tokens=100,
    temperature=0.7
)
if follow_up_answer:
```



```
print(f"Follow-up Answer: {follow_up_answer}")
```

### Example 3: JavaScript (Node.js) using `axios`

If your project is web-based.

```
javascript
const axios = require('axios');

const API_URL = 'https://api-ap-southeast-1.modelarts-maas.com/v1/chat/completions';
const AUTH_TOKEN = 'YOUR_ACTUAL_TOKEN';

async function callModel(modelName, userMessage, systemMessage = null, options = {}) {
  const messages = [];
  if (systemMessage) {
    messages.push({ role: 'system', content: systemMessage });
  }
  messages.push({ role: 'user', content: userMessage });

  const data = {
    model: modelName,
    messages: messages,
    ...options // Spread the additional options (max_tokens, temperature, etc.)
  };

  const config = {
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${AUTH_TOKEN}`
    },
    timeout: 60000
  };

  try {
    const response = await axios.post(API_URL, data, config);
    // Correctly extract the text from the verified response structure
    const generatedText = response.data.choices[0].message.content;
    return generatedText;
  } catch (error) {
```

```
console.error('Error calling API:');
if (error.response) {
  console.error('Status:', error.response.status);
  console.error('Data:', error.response.data);
} else {
  console.error('Message:', error.message);
}
throw error; // Re-throw the error after logging
}
}

// Usage Example
(async () => {
  try {
    const result = await callModel(
      'deepseek-v3.1',
      'Summarize the theory of relativity in one sentence.',
      null, // No system message
      { max_tokens: 50, temperature: 0.5 }
    );
    console.log('Model Output:', result);
  } catch (error) {
    // Handle errors in your application
  }
})();
```

## 4 Understanding the Response

A successful API call will return a JSON response with a structure similar to this:

```
{
  "id": "chatcmpl-123",
  "object": "chat.completion",
  "created": 1699871234,
  "model": "qwen3-32b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
```

```
        "content": "The capital of France is Paris."
    },
    "finish_reason": "stop"
}
],
"usage": {
    "prompt_tokens": 10,
    "completion_tokens": 5,
    "total_tokens": 15
}
}
```

- `choices[0].message.content`: This is the field you need to extract for the model's output.
- `finish_reason`: Why generation stopped. `"stop"` means the model hit a natural stop sequence.
- `usage`: Shows the number of tokens used, which may be important for rate limiting.

## 5 Best Practices & Tips for the Competition

### 1. Model Selection:

- **Qwen (32B)**: Likely more powerful and accurate for complex reasoning, coding, and detailed explanations. Use this for tasks requiring high quality. It might be slightly slower.
- **DeepSeek** : Faster and lighter. Use this for simpler text generation, classification, or when you need low latency and high throughput.

### 2. Parameter Tuning is Key: Don't just use the default parameters!

- For **factual, precise answers** (e.g., Q&A): Use **low** `temperature` (**~0.1-0.3**) and **high** `top_p` (**~0.9**).
- For **creative tasks** (e.g., story generation): Use **higher** `temperature` (**~0.7-0.9**).

- Adjust `max_tokens` carefully. Too low cuts off the response; too high wastes time and resources.
- 3. **Error Handling:** Always implement robust error handling in your code (as shown in the Python example) to manage API downtime, rate limits, or invalid requests.
- 4. **Cost & Rate Limits:** Be aware of any rate limits (requests per minute) imposed on the API. Structure your application to handle these limits gracefully (e.g., with retry logic).
- 5. **Prompt Engineering:** The quality of your `prompt` directly determines the quality of the output. Be clear and specific in your instructions. Provide examples if possible (few-shot learning).

## 6 Troubleshooting Common Issues

- `401 Unauthorized`: Your `X-Auth-Token` is missing, incorrect, or expired. Double-check with your competition organizers.
- `404 Not Found`: The endpoint URL is incorrect. Verify the exact URL.
- `413 Payload Too Large`: Your prompt is too long. Consider splitting it or providing a summary.
- `429 Too Many Requests`: You are being rate-limited. Slow down your requests.
- `500 Internal Server Error` or `503 Service Unavailable`: There is an issue on the ModelArts server side. Wait a moment and try again. If it persists, inform the competition support team.

## 7 Reference materials:

Tips: Due to the large size of the model, the invocation time may be long. You can use a streaming model or an 8B parameter model to avoid interface timeout. Alternatively, refer to the official DeepSeek documentation.

<https://api-docs.deepseek.com/>

## 8 Disclaimer:

1. This API is exclusively for developers to participate in the developer competition for testing and experience purposes and must not be used for any commercial production.
2. We are not responsible for safety and reliability during use.
3. This service model is stateless and does not record any user data.
4. The service will only be available during the competition period and will be closed after the competition ends.
5. The model provided in this competition is deployed on Huawei Cloud AI infrastructure located in Hong Kong. As a result, cross-regional data flow may occur. By using this service, participants are deemed to have acknowledged and agreed to such data transfer.
6. The organizing committee of the competition holds absolute management rights over this service. If any abuse or unwarranted resource consumption is detected, the right to use the service will be revoked.

**Good luck with your project! Use the power of these models wisely and innovatively.** For competition-specific issues (like token distribution), please contact the designated support channel.